Exercises — REST Routes in an Existing Backend

During the Lesson (In-class)

Exercise 1 — Trace the Request: Sequence Diagram for `GET /api/cars`

Individual | ~15 min

Before you add anything to the codebase, make sure you understand what happens when a request arrives.

Draw a sequence diagram that shows the full lifecycle of a request to `GET /api/cars`. Start from the moment the browser (or curl) sends the request, and end at the moment it receives the JSON response.

Include the following participants in your diagram:

```
Client → index.php → Kernel → Router → ApiController → CarRepository → Database
```

For each arrow (message), write:

- The method or action being called
- The return value travelling back

Use the following files as your source — read them, don't guess:

File	What to look at
`public/index.php`	How is the `Request` created? What does `$kernel->handle()` return?
`src/Kernel.php`	What does `handle()` do?
`src/Router.php`	How does `dispatch()` find the right route and call it?
`app/Controllers/ApiController.php`	What does `cars()` do step by step?
`app/Controllers/ApiBaseController.php`	What does `apiJson()` build and return?

Questions to answer after drawing the diagram:

1. At which point is the JSON envelope (meta / links / data) assembled?
2. Where is it decided that the response body should be JSON and not HTML?
3. If `CarRepository::all()` returns an empty array, what does the response look like?

Targets: making meaning / understanding the whole before modifying the parts

---

Exercise 2 — Add the Makes API Routes

Individual | ~25 min

The `MakeRepository` already exists and is registered in `ServiceProvider.php`. Your task is to expose it as a REST resource by following the exact same pattern used for Cars.

What to implement:

Route	Controller method	Expected response
`GET /api/makes`	`makes()`	Array of all makes, wrapped in the standard envelope
`GET /api/makes/{id}`	`make()`	Single make, or a 404 error envelope if not found

Steps:

1. Open `app/Controllers/ApiController.php` — add `makes()` and `make()` methods. Use `cars()` and `car()` as your template.
2. Open `app/RouteProvider.php` — register the two new routes. Use the existing `GET /api/cars` routes as your guide.
3. Inject `MakeRepositoryInterface` into `ApiController`'s constructor (look at how `CarRepositoryInterface` is already injected).
4. Test your routes:
   - `GET /api/makes` → should return a JSON list of all makes
   - `GET /api/makes/1` → should return a single make
   - `GET /api/makes/999` → should return a 404 error envelope

A Make has these fields: `id`, `name`, `country`, `description`

Extension (if you finish early): Nest the cars belonging to a make inside the `GET /api/makes/{id}` response, so the `data` object includes a `cars` array. You will need to look at how `CarRepository` fetches data and think about where to add a method that fetches cars by make ID.

Targets: application / transfer — applying the REST pattern to a new resource

---

After the Lesson (Homework / Follow-up)

Exercise 3 — Add the Categories API Routes

Individual | Due next lesson

Now do the same for Categories. The pattern is identical to Makes.

What to implement:

Route	What it returns
`GET /api/categories`	Array of all categories, in the standard envelope
`GET /api/categories/{id}`	Single category, or 404 envelope if not found

A Category has these fields: `id`, `name`

Follow the same steps as Exercise 2. When you are done, verify all four API routes work correctly: `/api/cars`, `/api/makes`, `/api/categories`, and the single-resource variants.

Targets: transfer — applying a learned pattern independently

Exercise 4 — Reflect and Argue

Individual | ~15 min | Written

Answer the following question in 150–250 words:

> A teammate suggests: *"Let's add a `/api/cars/search?q=mustang` endpoint. That's RESTful, right?"*
>
> Is this a good idea? Is it RESTful? What would you say to your teammate, and why?

There is no single correct answer. A strong response:

- Takes a clear position
- Uses at least one REST constraint (resource design, idempotence, uniform interface) to support it
- Acknowledges the trade-off or the counter-argument

Targets: deep learning / argumentation — applying theory to a design decision